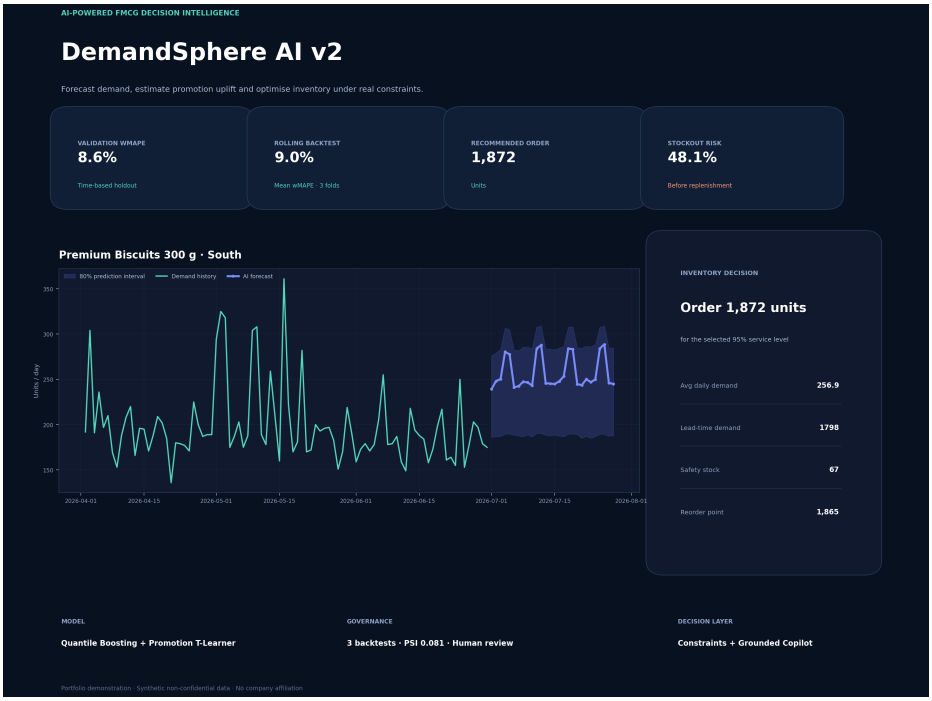
DATA SCIENCE INTERVIEW PREPARATION

DemandSphere AI

The complete end-to-end guide: from the business problem and raw data, through features, models and evaluation, to inventory decisions, monitoring, deployment and interview defence.

8.58% Holdout wMAPE	**56.5%** MAE gain vs lag-7 baseline	**77.1%** 80% interval coverage	**8.98%** Mean wMAPE, 3 rolling backtests
−1.50% Forecast bias	**0.081** Target PSI (stable)	**11,520** SKU-region-day rows	**24** Time series (6 SKUs × 4 regions)



The project's own dashboard preview (reports/demand_sphere_demo.png): Premium Biscuits 300 g, South region, 10% promotion, 1,800 units in stock.

How to use this guide

Tonight (2–3 hours): read Sections 1–4 for the story, 5–13 for the mechanics, then Section 17 (weak spots) and 18 (question bank) twice.

Tomorrow morning (30 minutes): read the one-page cheat sheet (Section 20) and answer the self-test aloud.

Every number in this guide comes from the project's actual artifacts (metrics.json, rolling_backtests.csv, promotion_uplift.csv, validation_predictions.csv).

Contents

1. The project in one page 5

1.1 The 30-second pitch (memorise) 5

1.2 The 2-minute story (structure for “walk me through your project”) 5

1.3 What makes it a strong project (your selling points) 5

2. The business problem and domain vocabulary 6

2.1 Why demand forecasting matters in FMCG 6

2.2 Vocabulary you must use fluently 6

3. Tech stack and repository map 7

3.1 Tech stack 7

3.2 Repository map 7

3.3 How to run it 8

4. End-to-end architecture 9

4.1 Offline: what python train.py does, in order 9

4.2 Online: what happens when a planner clicks “Run AI scenario” 9

5. Data: generation, censoring and correction 11

5.1 What the dataset contains 11

5.2 The demand formula (how the generator thinks) 11

5.3 Columns 11

5.4 Censored demand and lost-sales correction (a key talking point) 12

5.5 Data validation 12

6. Feature engineering and leakage control 13

6.1 The 23 features 13

6.2 Cyclical encoding 13

6.3 Leakage prevention — the single most important line 13

6.4 Time-based split 13

7. The model: gradient boosting and quantile regression 15

7.1 Gradient boosting from first principles 15

7.2 Why this model and not something else 15

7.3 The pipeline and hyperparameters 15

7.4 Three models: point, P10 and P90 16

8. Evaluation: metrics, baseline, backtests 17

8.1 Metric definitions 17

8.2 Results on the 56-day holdout (2026-05-06 → 2026-06-30) 17

8.3 Why compare with a lag-7 (seasonal naive) baseline? 17

8.4 Interval coverage: reading 77% 18

8.5 Rolling-origin backtests 18

9. Explainability 20
9.1 Permutation importance (global) 20
9.2 Local counterfactual explanation 20
10. Forecasting the future: the scenario service 21
10.1 The problem 21
10.2 Assumed future inputs 21
11. From forecast to decision: inventory optimisation 22
11.1 The policy: periodic review, order-up-to 22
11.2 Why each piece 22
11.3 Worked example (the dashboard's default scenario) 22
11.4 Tested business properties 23
12. Promotion uplift with a T-learner 24
12.1 The question and the trap 24
12.2 The T-learner (Two-model learner) 24
12.3 Sanity check against the known truth 24
13. Monitoring and governance 26
13.1 Population Stability Index (PSI) 26
13.2 Isolation Forest anomaly detection 26
13.3 Retraining and promotion gates 26
13.4 Model card 26
14. The Planning Copilot 27
15. Serving, testing and delivery 28
15.1 FastAPI (api.py) 28
15.2 Gradio dashboard (app.py) 28
15.3 Tests (tests/test_core.py) 28
15.4 CI, Docker, reproducibility 28
16. The 5-minute live demo script 29
17. Critical review: weak spots and how to answer them 30
18. Interview question bank 31
A. Problem and data 31
B. Features and leakage 31
C. Modelling 31
D. Evaluation 32
E. Inventory and business 32
F. Promotion and causality 33
G. MLOps and production 33
H. Behavioural 33
19. Glossary of concepts 34

20. One-page cheat sheet and self-test 35

Numbers to memorise 35

Key formulas 35

Self-test — answer aloud without looking 35

Appendix: resume bullets (accurate wording) 36

1. The project in one page

1.1 The 30-second pitch (memorise)

> **Say this**
> "I built DemandSphere AI, an end-to-end demand forecasting and inventory decision system for an FMCG business. It forecasts daily demand for every product in every region, with an 80% uncertainty band, and turns that forecast into a concrete order quantity that respects case packs, minimum order, warehouse capacity and shelf life. On a 56-day period the model never saw, it reached 8.58% weighted MAPE, 56% better than a 'same-day-last-week' baseline, and it held up across three rolling backtests. I also estimated promotion uplift with a T-learner, added drift and anomaly monitoring, and served it through FastAPI and a Gradio dashboard with tests, CI and Docker."

1.2 The 2-minute story (structure for "walk me through your project")

1. **Problem.** FMCG companies lose money two ways: stockouts (lost sales, brand switching) and overstock (tied-up cash, expiry). Planners need a forecast *and* an action.
2. **Data.** 11,520 daily records — 6 SKUs × 4 regions × 480 days — with price, discount, promotion, marketing, competitor price, weather, holidays, stock and sales. Synthetic, so it is safe to share, but built with realistic demand drivers.
3. **Key data insight.** Sales ≠ demand. When stock runs out, sales are censored. I corrected those days to estimated true demand before training.
4. **Features.** Lags (1, 7, 14, 28 days), rolling means and volatility, calendar and cyclical encodings, commercial and external drivers — all shifted by one day so nothing leaks from the future.
5. **Model.** Histogram gradient boosting: one model for the expected value and two quantile models (P10, P90) for an 80% interval.
6. **Evaluation.** Strict time split; last 56 days held out. 8.58% wMAPE, −1.5% bias, 56.5% lower MAE than lag-7 baseline, 3 rolling backtests averaging 8.98%.
7. **Decision.** Forecast → lead-time demand → safety stock → reorder point → order-up-to quantity, rounded to case packs and capped by capacity and shelf-life.
8. **Extras.** T-learner promotion uplift, permutation importance, counterfactual explanations, PSI drift, Isolation Forest anomalies, a grounded rule-based copilot.
9. **Engineering.** FastAPI, Gradio, unit tests, GitHub Actions CI, Docker, saved and versioned artifacts.

1.3 What makes it a strong project (your selling points)

- It closes the **decision loop** — prediction → uncertainty → action — instead of stopping at a metric in a notebook.
- It is **leakage-aware**: shifted features, time-based split, rolling-origin backtests.
- It is **probabilistic**: quantile models give a range, not just a point.
- It is **honest**: baseline comparison, bias reporting, a model card, and explicit non-causal disclaimers on promotion uplift.
- It is **deployable**: API with validation, UI, tests, CI, Docker.

2. The business problem and domain vocabulary

2.1 Why demand forecasting matters in FMCG

FMCG (fast-moving consumer goods) companies sell cheap, high-volume products — biscuits, soap, flour, beverages — through many regions and channels. Margins are thin, so inventory mistakes are expensive.

Mistake	What happens	Business cost
Under-forecast	Shelf is empty when a customer arrives (stockout)	Lost revenue, customer switches brand, retailer penalties
Over-forecast	Too much stock sits in the warehouse	Working capital locked up, storage cost, expiry and write-offs

Therefore the real question is not “how many units tomorrow?” but **“how much should I order today, and how sure am I?”** That is why the project has three layers: a point forecast, an uncertainty band, and an inventory decision.

2.2 Vocabulary you must use fluently

Term	Meaning in this project
SKU	Stock Keeping Unit — one sellable product, e.g. FOOD-BISC-300G (Premium Biscuits 300 g).
Series / grain	One SKU in one region. 6 × 4 = 24 daily time series. Forecast grain = SKU-region-day.
Channel	General Trade (kirana stores), Modern Trade (supermarkets), E-commerce. Each SKU-region has a fixed channel here.
Lead time	Days between placing an order with the supplier and the stock arriving (default 7).
Review period	How often stock is reviewed and an order can be placed (7 days in the code).
Service level	Target probability of NOT stocking out during the lead time (default 95%).
Safety stock	Extra buffer held to absorb forecast error.
Reorder point (ROP)	Stock level at which you should place an order = lead-time demand + safety stock.
Case pack	Units per shipping case (12). Orders must be multiples of this.
MOQ	Minimum order quantity the supplier accepts.
Stockout / censoring	When stock hits zero, sales stop even if customers still want the product, so observed sales understate demand.
Promotion uplift	Extra demand caused by a promotion compared with no promotion.
wMAPE	Weighted mean absolute percentage error — the standard retail forecast-accuracy metric.

3. Tech stack and repository map

3.1 Tech stack

Layer	Tool	Why
Data handling	pandas, NumPy	Group-wise time-series transforms (shift, rolling) are natural in pandas
Modelling	scikit-learn HistGradientBoostingRegressor, IsolationForest, Pipeline, ColumnTransformer, OneHotEncoder	Fast, robust tabular ML; quantile loss built in; pipelines keep preprocessing and model together
Explainability	sklearn.inspection.permutation_importance	Model-agnostic, measured on held-out data
Persistence	joblib	Saves the whole model bundle as one artifact
API	FastAPI + Pydantic + Uvicorn	Typed request validation, automatic Swagger docs at /docs
UI	Gradio + Plotly	Interactive what-if dashboard in pure Python
Quality	unittest, GitHub Actions, compileall	Automated checks on every push
Packaging	Docker (python:3.12-slim), Makefile	Reproducible runtime; one-command tasks

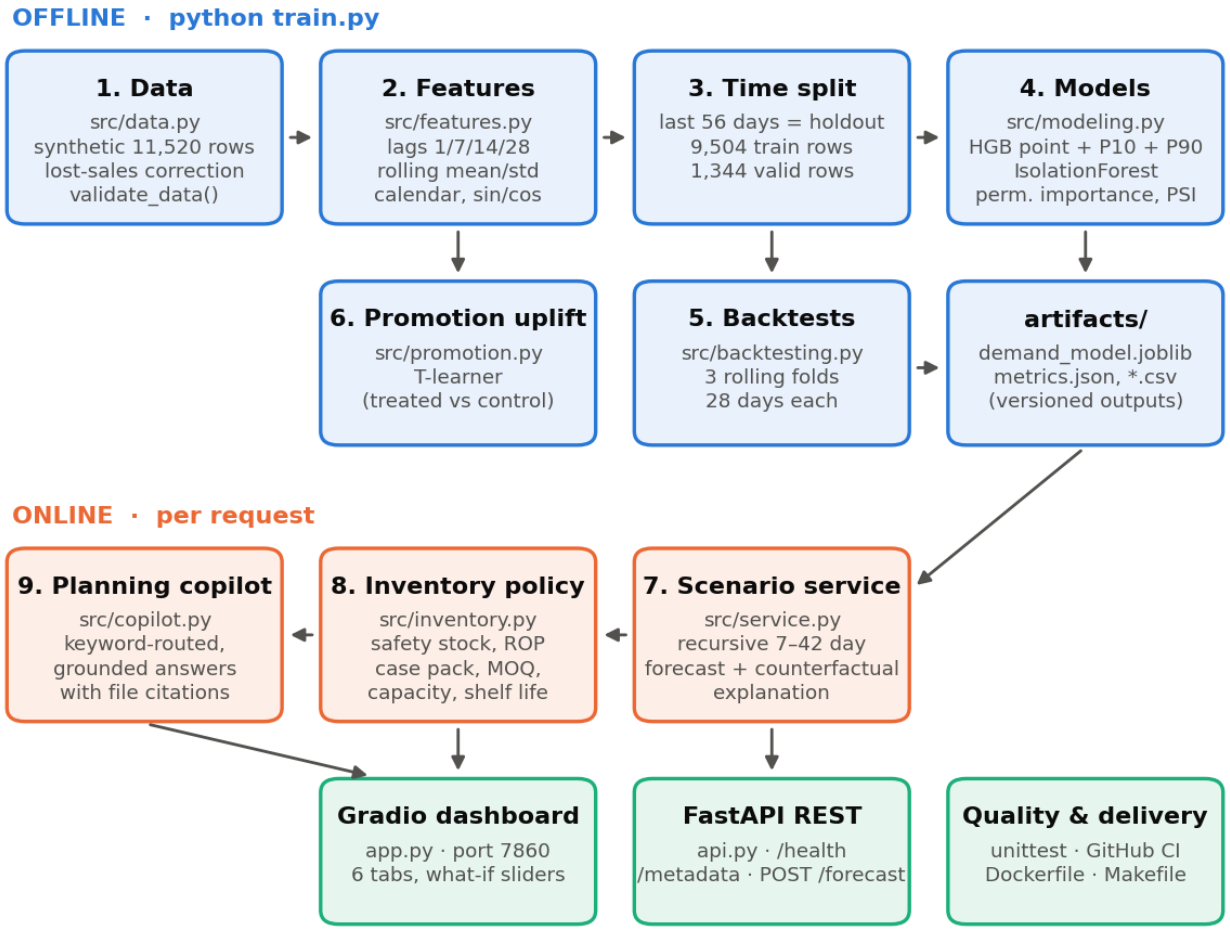
3.2 Repository map

File	Responsibility
train.py	Orchestrates everything: generate data → train and save models → rolling backtests → promotion uplift. Prints headline metrics.
src/config.py	Paths, RANDOM_STATE = 42, FORECAST_HORIZON = 56, the feature lists and the target name (units_demand).
src/data.py	Synthetic data generator, stockout censoring, lost-sales correction, load_data(), validate_data().
src/features.py	engineer_features() (lags, rolling, calendar) and time_split().
src/modeling.py	_pipeline(), DemandModelBundle, regression_metrics(), population_stability_index(), train_and_save(), load_bundle().
src/backtesting.py	rolling_origin_backtest(): 3 folds × 28 days.
src/promotion.py	estimate_adjusted_promotion_uplift(): the T-learner.
src/service.py	DemandIntelligenceService: recursive multi-day forecast, scenario counterfactual, explanation text.
src/inventory.py	inventory_policy(): safety stock, reorder point, order quantity, constraints, stockout risk.
src/copilot.py	PlanningCopilot: keyword-routed answers grounded in saved artifacts.
app.py	Gradio dashboard (6 tabs) on port 7860.
api.py	FastAPI app: GET /health, GET /metadata, POST /forecast.
tests/test_core.py	7 tests: data contract, leakage, inventory monotonicity, constraints, interval ordering, beats baseline, copilot.
artifacts/	demand_model.joblib, metrics.json, feature_importance.csv, validation_predictions.csv, rolling_backtests.csv, promotion_uplift.csv.
docs/	ARCHITECTURE.md, MODEL_CARD.md, INTERVIEW_GUIDE.md.
.github/workflows/ci.yml, Dockerfile, Makefile	CI pipeline, container image, shortcuts (make train / test / demo / api).

3.3 How to run it

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python train.py                      # generates data, trains, backtests, uplift -> artifacts/
python app.py                        # dashboard at http://127.0.0.1:7860
uvicorn api:app --reload             # API; Swagger at http://127.0.0.1:8000/docs
python -m unittest discover -s tests -v
docker build -t demand-sphere-ai . && docker run -p 7860:7860 demand-sphere-ai
```

4. End-to-end architecture



Offline training pipeline (blue), online request path (orange), and serving and quality layers (green).

4.1 Offline: what python train.py does, in order

1. **generate_fmcg_data()** builds 11,520 rows and writes data/sample_fmcg_sales.csv.
2. **train_and_save()** engineers features, splits by time, trains the point, P10 and P90 models, computes metrics against the lag-7 baseline, fits the Isolation Forest on validation residuals, computes permutation importance and PSI, and saves the bundle and CSVs.
3. **rolling_origin_backtest()** retrains the point model three times on expanding history and scores three consecutive 28-day windows.
4. **estimate_adjusted_promotion_uplift()** trains the treated and control models and writes per-SKU uplift.

4.2 Online: what happens when a planner clicks “Run AI scenario”

1. app.py (or POST /forecast) calls **service.run_scenario(sku, region, horizon, promotion, discount, stock, lead time, …)**.
2. The service filters history for that SKU-region and forecasts day by day **recursively** for the horizon (7–42 days).
3. The forecast goes into **inventory_policy()**, which returns safety stock, reorder point, recommended order, stockout risk and binding constraints.
4. The service builds a **counterfactual** (same day, no promotion) to explain the promotion's effect, and lists the top global features.

5. The dashboard shows a chart with the history, forecast and 80% band, the policy table and explanation; the copilot can answer follow-up questions.

> **Interview framing**
> “Training is a batch job that produces versioned artifacts; serving is stateless and loads the saved bundle. That separation is what lets the same model power both a UI and an API.”

5. Data: generation, censoring and correction

5.1 What the dataset contains

The data is **synthetic** (fictional) on purpose — no confidential company data — but it is generated from a realistic multiplicative demand model, so the patterns an ML model must learn are the ones real FMCG data has.

SKU	Product	Category	List price	Base units/day
FOOD-ATTA-5KG	Whole Wheat Flour 5 kg	Staples	286	74
FOOD-BISC-300G	Premium Biscuits 300 g	Foods	62	128
FOOD-NOOD-280G	Instant Noodles 280 g	Foods	86	112
PC-SOAP-125G	Bathing Soap 125 g	Personal Care	54	96
BEV-FRUIT-1L	Fruit Beverage 1 L	Beverages	118	83
HOME-INC-100	Incense Sticks 100 pack	Home Care	78	68

Regions and their demand multipliers: North 1.06, South 1.15, East 0.94, West 1.02. Date range: 2025-03-08 to 2026-06-30 (480 days). Rows = 6 SKUs × 4 regions × 480 days = **11,520**.

5.2 The demand formula (how the generator thinks)

```
expected = base × region × annual_season × weekend × trend × promo_uplift × festival_uplift
           × weather_uplift × competitor_index^0.8 × (1 + marketing / 25000)

annual_season   = 1 + 0.09·sin(2π·dayofyear/365.25)          # ±9% yearly cycle
weekend         = 1.12 on Sat/Sun                              # +12%
trend           = 1 + 0.00032·step                             # ≈ +15% over 480 days
promo_uplift    = 1 + (0.28 + 0.012·discount%)·promotion        # 10% discount -> +40%
festival_uplift = 1 + 0.22·festival   (4 smooth Gaussian peaks a year)
weather         = Beverages: +2.5% per °C above 29, −0.4% per mm rain; Home Care: extra festival boost
latent_demand   = Normal(expected, sd = max(5, 10% of expected))   # the random noise
```

- **Promotions** happen on about 14% of days, more often Friday–Sunday (11% base + 6% on Fri/Sat/Sun). Discount is 5–25%.
- **Price** = list price × (1 − discount) × small random noise. **Marketing spend** ≈ 1,450 on promo days vs 520 otherwise.
- **Holiday flag** = 1 when the festival signal is above 0.48 (about ±10 days around each peak).
- **Stockouts**: on about 3.5% of days, opening stock is forced to 40–88% of demand, so `units_sold = min(demand, stock)`.

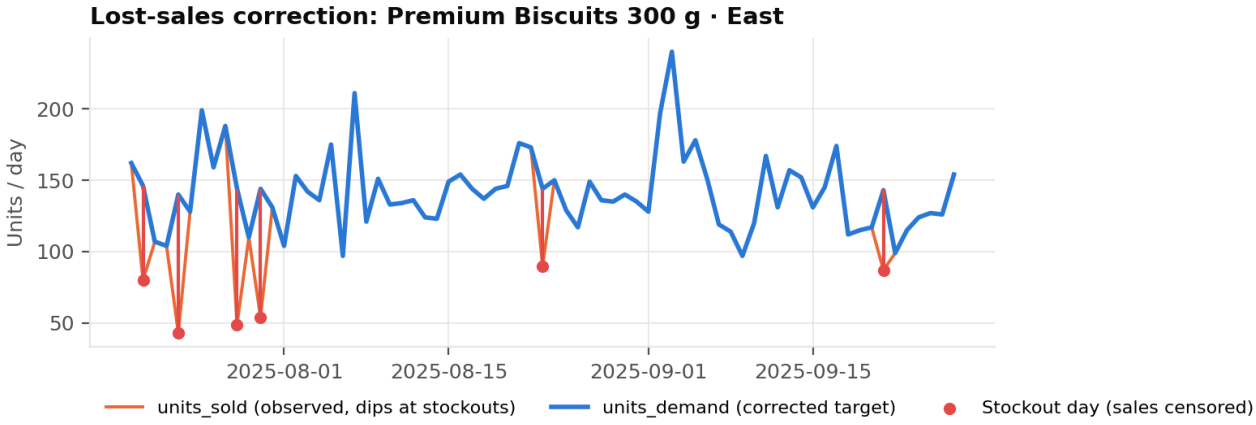
5.3 Columns

Column	Description
date, sku, product_name, category, region, channel	Keys and hierarchy
price, discount_pct, promotion	Commercial levers (promotion = 1 exactly when discount_pct > 0)
marketing_spend, competitor_price_index	Demand drivers
holiday, temperature_c, rainfall_mm	External drivers
opening_stock, units_sold, stockout_flag	Inventory and observed sales
units_demand	**The target**: sales corrected for stockouts

5.4 Censored demand and lost-sales correction (a key talking point)

If the shelf is empty, sales stop even though customers still want the product. Observed sales are therefore **right-censored**: they are a lower bound on demand. Training on raw sales teaches the model that demand falls whenever stock runs out, which biases forecasts down and then causes even more stockouts — a vicious cycle.

```
clean_sales  = units_sold where stockout_flag == 0                # trust only uncensored days
history_mean = clean_sales.groupby([sku, region]).shift(1).rolling(28, min_periods=7).mean()
fallback     = clean_sales.groupby([sku, region]).median()
estimate     = history_mean or fallback or units_sold
units_demand = max(units_sold, estimate) if stockout_flag == 1 else units_sold
```



On stockout days (red), observed sales collapse; the corrected target (blue) restores an estimate of true demand.

Talking point

“Sales are not demand. I treated stockout days as censored observations and replaced them with an estimate from the previous 28 clean days. The estimate itself uses shift(1), so it is also leakage-free.”

Limitation to admit

The correction ignores whether the stockout day was a promotion or a weekend, so it can underestimate demand on those days. Better options: model-based imputation (predict demand with the trained model), or a censored-regression / Tobit-style likelihood.

5.5 Data validation

`validate_data()` checks that required columns exist, the frame is not empty, demand is never negative and price is always positive. It is a lightweight **data contract**; in production you would add tools like Great Expectations or Pandera, plus freshness, null-rate and unseen-category checks.

6. Feature engineering and leakage control

6.1 The 23 features

Group	Features	Intuition
Hierarchy (categorical)	sku, category, region, channel	Different base levels; one-hot encoded
Commercial	price, discount_pct, promotion, marketing_spend, competitor_price_index	Levers the business controls or reacts to
External	holiday, temperature_c, rainfall_mm	Festivals and weather move demand
Calendar	day_of_week, month, day_sin, day_cos	Weekly and monthly seasonality
Lags	lag_1, lag_7, lag_14, lag_28	Demand 1, 7, 14, 28 days ago (same weekday for 7, 14, 28)
Rolling stats	rolling_mean_7, rolling_mean_28, rolling_std_7	Recent level and recent volatility

6.2 Cyclical encoding

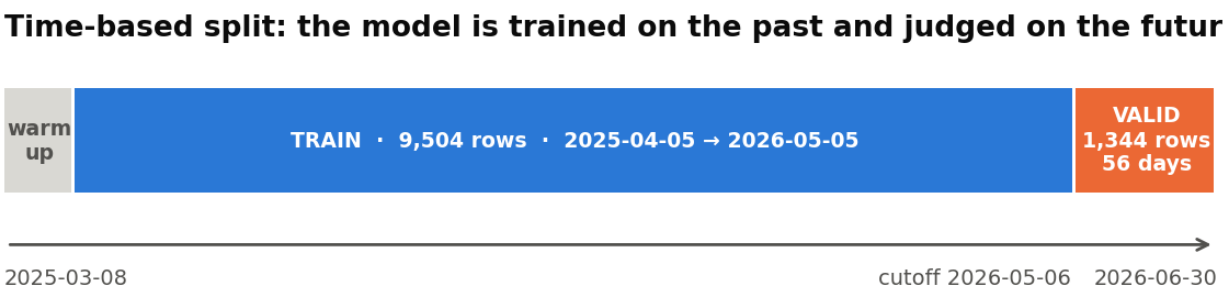
Day of week as 0–6 makes Sunday (6) look far from Monday (0). Mapping it onto a circle with `sin(2π·dow/7)` and `cos(2π·dow/7)` puts them next to each other. Tree models can split on the raw integer too, so both are kept.

6.3 Leakage prevention — the single most important line

```
grouped = frame.groupby(["sku", "region"], sort=False)["units_demand"]
for lag in (1, 7, 14, 28):
    frame[f"lag_{lag}"] = grouped.shift(lag)
frame["rolling_mean_7"] = grouped.transform(lambda s: s.shift(1).rolling(7, min_periods=7).mean())
```

- **groupby(sku, region)** stops a lag from crossing from one product's series into another's.
- **shift(1) before rolling** means the 7-day mean for date t uses days t−7 … t−1, never day t itself. Without the shift, the target leaks into its own feature.
- The first 28 days of each series have missing lags and are **dropped** (480 − 28 = 452 usable days × 24 series = 10,848 rows).
- A unit test (`test_lag_features_only_use_prior_dates`) checks that lag_1 equals the previous day's target and that the target is not in FEATURES.

6.4 Time-based split




```
cutoff = frame["date"].max() - pd.Timedelta(days=validation_days - 1)   # 2026-05-06
train  = frame[frame["date"] <  cutoff]     # 9,504 rows (396 days × 24)
valid  = frame[frame["date"] >= cutoff]     # 1,344 rows (56 days × 24)
```

Why not a random split?

A random split mixes future days into training. The model would effectively see the future (and neighbouring days share lag values), giving optimistic metrics that collapse in production. Forecasts must always be tested forward in time.

7. The model: gradient boosting and quantile regression

7.1 Gradient boosting from first principles

1. Start with a constant prediction (for example the mean demand).
2. Compute the residuals — more precisely, the negative gradient of the loss with respect to the current prediction. For squared error, that is just actual − predicted.
3. Fit a small decision tree to those residuals.
4. Add the tree's output, scaled by a learning rate (0.055), to the current prediction.
5. Repeat for up to 260 trees, or until early stopping says the validation loss stopped improving.

Each tree corrects what the previous trees got wrong. A small learning rate plus many trees gives a smooth, well-regularised fit. **Histogram** gradient boosting (the LightGBM idea, implemented in scikit-learn) first buckets every numeric feature into at most 255 bins, so finding the best split is a fast scan over bins instead of over every raw value.

7.2 Why this model and not something else

Option	Why not chosen as the main model
Linear regression	Demand effects are multiplicative and interact (promotion × category × weekday). It would need many hand-made interaction terms.
ARIMA / ETS / Prophet	Fit one series at a time and handle many external drivers poorly; cannot share learning across 24 series.
Random forest	Usually less accurate than boosting on tabular data; no built-in quantile loss in scikit-learn.
LSTM / Transformer	Too little data (24 series × ~450 days), slower, harder to explain; complexity must earn its cost.
HistGradientBoosting (chosen)	**Captures non-linear interactions, fast, native quantile loss, one global model across all series, easy to explain.**

One **global model** is trained across all 24 series, with SKU, region and category as features. It learns shared patterns (for example, weekends lift everything) and still allows series-specific levels.

7.3 The pipeline and hyperparameters

```
preprocessing = ColumnTransformer([
    ("category", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CATEGORICAL_FEATURES),
    ("numeric",  "passthrough", NUMERIC_FEATURES)])
model = HistGradientBoostingRegressor(loss=loss, quantile=quantile, learning_rate=0.055, max_iter=260,
    max_leaf_nodes=24, min_samples_leaf=28, l2_regularization=0.8,
    early_stopping=True, validation_fraction=0.12, random_state=42)
Pipeline([("preprocess", preprocessing), ("model", model)])
```

Parameter	Value	Effect
learning_rate	0.055	Size of each tree's contribution; smaller is smoother but needs more trees
max_iter	260	Maximum number of trees
max_leaf_nodes	24	Limits each tree's complexity (depth of interactions)
min_samples_leaf	28	Each leaf needs at least 28 rows, which prevents fitting noise
l2_regularization	0.8	Shrinks leaf values; reduces overfitting

Parameter	Value	Effect
early_stopping / validation_fraction	True / 0.12	Holds out 12% of training rows and stops when they stop improving
handle_unknown='ignore'	—	A new SKU at inference does not crash the encoder (all its one-hot columns become 0)

Why a Pipeline? Preprocessing is fitted only on training data and saved together with the model, so training and serving always apply identical transformations (no **train/serve skew**).

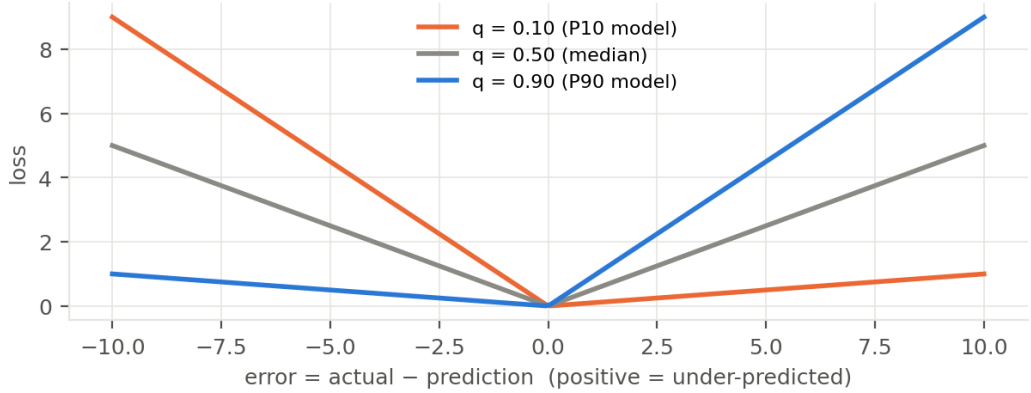
7.4 Three models: point, P10 and P90

The project trains the same pipeline three times with different losses:

Model	Loss	Meaning
point_model	squared_error	Expected (mean) demand
lower_model	quantile, q = 0.10	Demand is below this about 10% of the time
upper_model	quantile, q = 0.90	Demand is below this about 90% of the time

Together P10 and P90 form an **80% prediction interval**. Quantile models are trained with the **pinball loss**: for quantile q, an under-prediction costs q × error and an over-prediction costs (1 − q) × |error|. For q = 0.9, under-predicting is 9× more expensive than over-predicting, so the model pushes its prediction up until only about 10% of actual values lie above it.

Pinball (quantile) loss: P90 punishes under-prediction 9× more than over-prediction



Post-processing: predictions are clipped at 0 (demand cannot be negative), and the three outputs are sorted so that lower ≤ point ≤ upper always holds. Separately trained quantile models can otherwise cross each other.

```
point = np.maximum(0.0, point_model.predict(X))
low   = np.minimum.reduce([raw_low, point, raw_high])
high  = np.maximum.reduce([raw_low, point, raw_high])
```

8. Evaluation: metrics, baseline, backtests

8.1 Metric definitions

Metric	Formula	Why it is used
MAE	mean(|actual − pred|)	Error in units; planners understand it directly
RMSE	sqrt(mean((actual − pred)^2))	Penalises large misses more; RMSE much bigger than MAE signals occasional big errors
MAPE	mean(|error| / max(actual, 1))	Percentage error per row; explodes near zero demand
wMAPE	Σ|error| / Σ actual	Volume-weighted; the retail standard; robust to low-volume rows
Bias	Σ(pred − actual) / Σ actual	Systematic over (+) or under (−) forecasting; matters for inventory
Coverage	share of actuals inside [P10, P90]	Checks whether the uncertainty band is honest (target 80%)

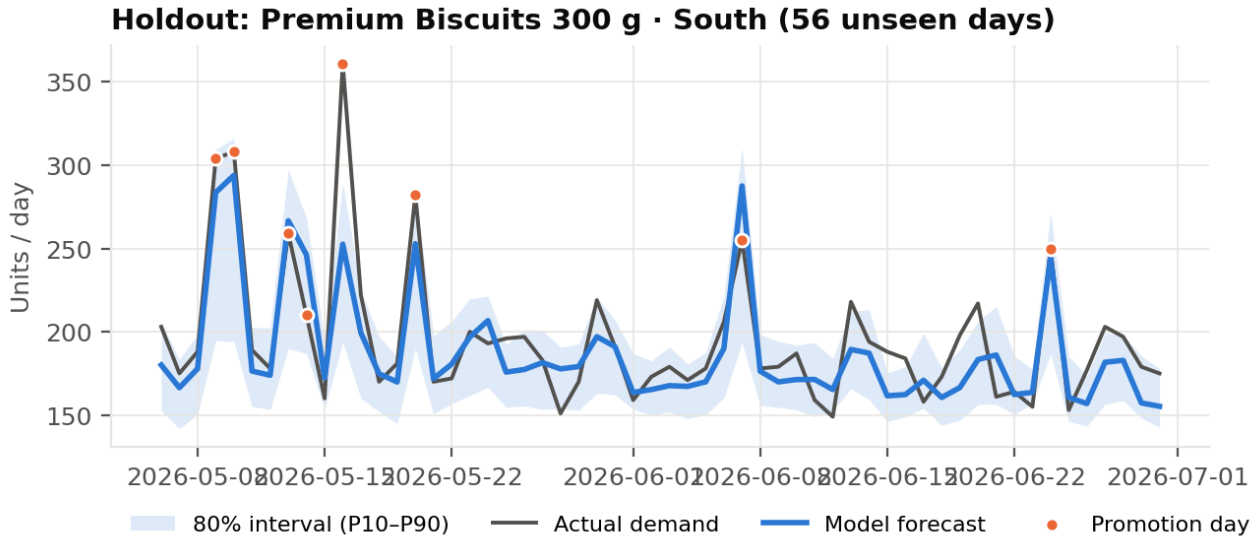
8.2 Results on the 56-day holdout (2026-05-06 → 2026-06-30)

Metric	Model	Lag-7 baseline	Read it as
MAE	11.53	26.52	Model is off by about 11.5 units per day on average
RMSE	15.38	37.66	Few large misses
MAPE	8.58%	19.24%	—
wMAPE	**8.58%**	19.74%	About 8.6 units wrong per 100 units sold
Bias	−1.50%	+0.55%	Slight systematic under-forecast
MAE improvement	**56.51%**	—	(26.52 − 11.53) / 26.52
80% interval coverage	77.08%	—	Slightly too narrow (target 80%)

Average daily demand in the holdout is about 134 units, so MAE 11.5 ≈ 8.6% of the average — consistent with wMAPE. Accuracy is similar across categories (Beverages 8.2% … Personal Care 9.2%), and slightly worse on promotion days (9.0% vs 8.5%).

8.3 Why compare with a lag-7 (seasonal naive) baseline?

“Demand will equal the same weekday last week” costs nothing to build and captures weekly seasonality. If a machine-learning model cannot beat it, it adds no value. Beating it by 56.5% shows the model learns promotions, trends and drivers that the naive rule cannot. **Always present a model relative to a baseline.**



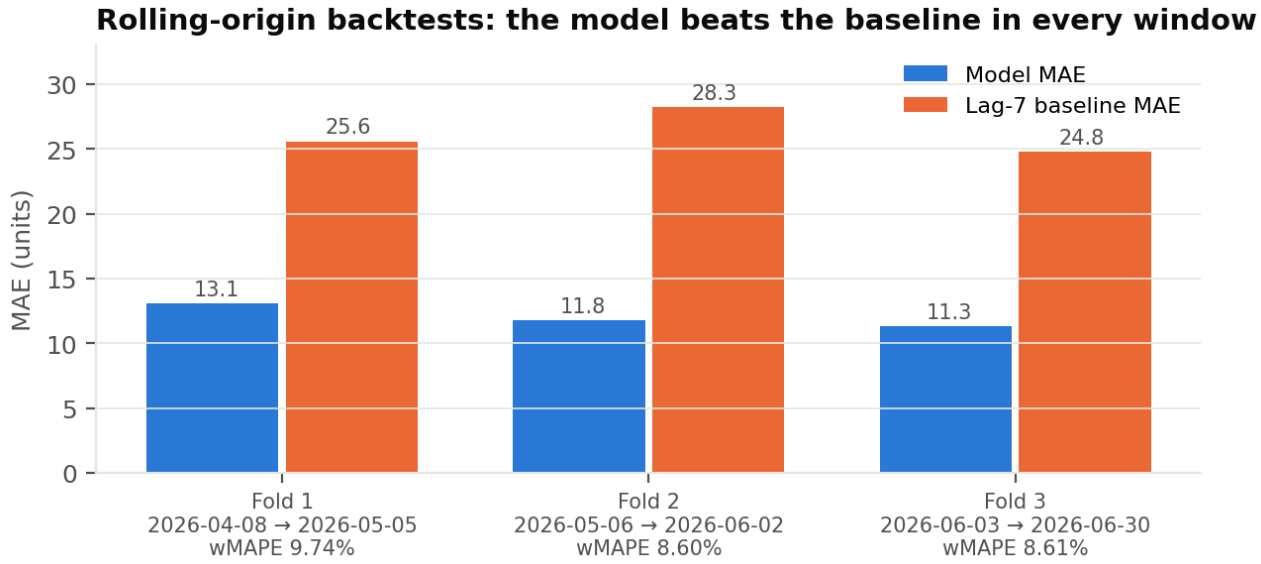
One series in the holdout. The forecast tracks promotion spikes (orange dots); most actuals stay within the 80% band.

8.4 Interval coverage: reading 77%

Of holdout actuals, 9.4% fall below P10 and 13.5% above P90 (ideal: 10% and 10%). The upper side is too low — consistent with the −1.5% bias. Quantile models are rarely perfectly calibrated on a finite holdout. Fix: **conformal calibration** — widen the band using residual quantiles from a recent calibration window, ideally per category.

8.5 Rolling-origin backtests

A single split can be lucky. The backtest retrains the model three times on expanding history and tests each on the next 28 days.



Fold	Window	wMAPE	MAE	Bias	Baseline MAE	Improvement
1	2026-04-08 → 05-05	9.74%	13.13	−4.07%	25.60	48.7%
2	2026-05-06 → 06-02	8.60%	11.79	−1.39%	28.25	58.3%
3	2026-06-03 → 06-30	8.61%	11.33	−1.73%	24.78	54.3%
Mean		**8.98%**	12.08	−2.40%	26.21	53.8%

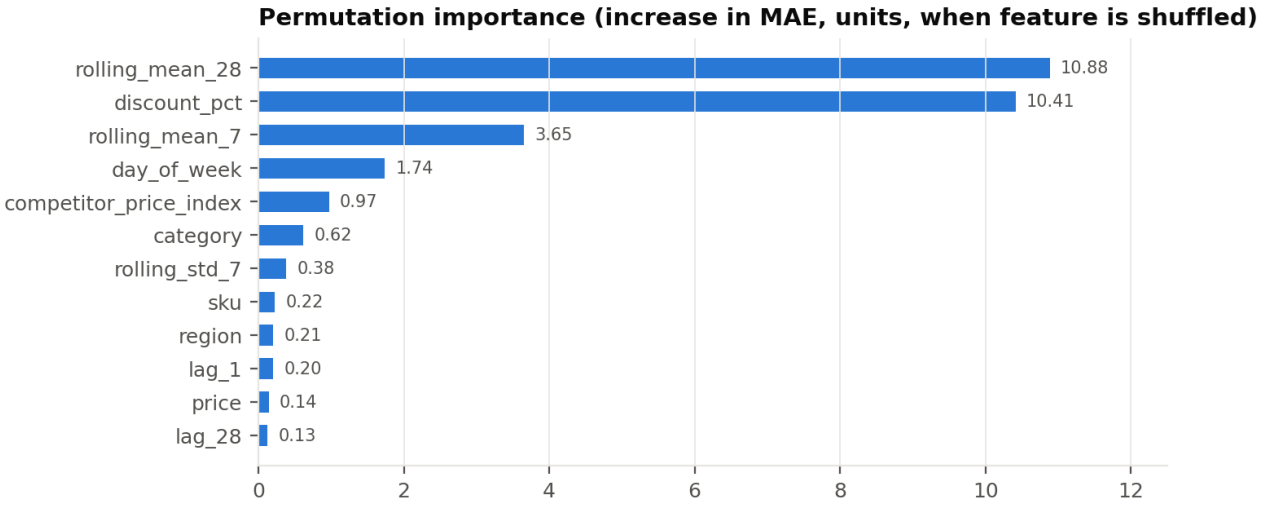
Talking point

“Accuracy is stable across three consecutive months and always well above the baseline. The model is consistently slightly under-forecasting, which matters for inventory, so I report bias as a first-class metric.”

9. Explainability

9.1 Permutation importance (global)

Algorithm: score the trained model on the holdout (MAE). Then, for one feature at a time, randomly shuffle that column, re-score, and record how much MAE got worse. Repeat 3 times and average. Because it is measured on held-out data and works at the level of raw columns (before one-hot encoding), it is model-agnostic and easy to interpret: **importance = extra units of MAE when that feature becomes noise**.



- **rolling_mean_28 (10.9)**: the recent demand level. Shuffling it roughly doubles MAE (11.5 → about 22.4).
- **discount_pct (10.4)**: promotions are the biggest driver of day-to-day spikes.
- **rolling_mean_7 (3.6)** and **day_of_week (1.7)**: short-term level and the weekend effect.
- lag_7 is low because rolling means already carry the same information; correlated features share credit.

Why do promotion and holiday score exactly 0? (a likely trick question)

promotion is fully redundant with discount_pct (promotion = 1 exactly when discount > 0). The trees split on discount_pct, so shuffling promotion changes nothing.

holiday is never 1 inside the May–June holdout window (no festival peak falls there), so shuffling a constant column changes nothing. It does not mean holidays don't matter.

General lesson: permutation importance is **conditional on the evaluation data** and **underestimates correlated features**. For per-prediction explanations, use SHAP.

9.2 Local counterfactual explanation

For every scenario the service predicts the first forecast day twice: once as configured and once with promotion = 0, discount = 0 and the normal price. The difference is reported as “the selected promotion changes day-one demand by +X% versus a no-promotion counterfactual.” This answers the planner's real question: *what does this lever do?*

10. Forecasting the future: the scenario service

10.1 The problem

To forecast day 2 you need lag_1 = demand on day 1, which has not happened yet. The service solves this with **recursive (iterated) forecasting**.

```
demand_history = history["units_demand"].tolist()
for date in next `horizon` days:
    row = build_feature_row(history, demand_history, date, promotion, discount)   # lags from demand_history
    point, low, high = bundle.predict(row)
    demand_history.append(point)        # feed the prediction back as if it were observed
```

10.2 Assumed future inputs

Feature	Value used for future days
price	Median non-promo price × (1 − discount)
marketing_spend	1,450 if promotion else 520 (the generator's typical values)
competitor_price_index	Median of the last 60 days
temperature_c / rainfall_mm	Mean / median of the last 30 days
holiday	A simple placeholder rule (Sunday on the 1st, 15th or 30th)
lags and rolling stats	Computed from actual history plus the forecasts appended so far

Trade-offs to be ready for

Error accumulation: in recursive forecasting, a mistake on day 1 feeds into day 2's lags, and so on. Alternatives: **direct** forecasting (a separate model per horizon) or multi-output models.

Holdout vs production mismatch: the 8.58% holdout score is **one-step-ahead** (lags use real past values). The dashboard's 28-day recursive forecast will be less accurate, especially later in the horizon. The fix is a recursive backtest reporting error by horizon.

Exogenous assumptions: in production, temperature would come from a weather forecast, and holiday and marketing from the promotion calendar. The holiday rule here does not match the training definition.

11. From forecast to decision: inventory optimisation

11.1 The policy: periodic review, order-up-to

Every review period (7 days) the planner looks at stock and orders enough to bring it up to a **target level** that covers demand until the *next* order arrives (lead time + review period) plus a safety buffer.

```
z               = NormalDist().inv_cdf(service_level)          # 95% -> 1.645
lead_time_demand = avg_daily_forecast × lead_time
safety_stock     = z × residual_std × sqrt(lead_time)
reorder_point    = lead_time_demand + safety_stock
target_stock     = avg_daily_forecast × (lead_time + review_period) + safety_stock
order            = ceil((target_stock − current_stock) / case_pack) × case_pack
order            = max(order, MOQ rounded to case pack)    if 0 < order < MOQ
order            = min(order, warehouse capacity − stock, rounded down to case pack)
order            = min(order, 0.85 × avg_daily × shelf_life − stock, rounded down)
stockout_risk    = P(lead-time demand > current_stock) = 1 − Φ((stock − LT demand) / (σ·√L))
```

11.2 Why each piece

- **z-score:** under a normal error assumption, holding z standard deviations of buffer gives that service level (1.645 → 95%, 2.33 → 99%). The service level is clipped to 80–99.5%.
- **σ (residual_std):** the standard deviation of holdout forecast errors, about 15.25 units. Safety stock protects against *forecast error*, not against demand itself.
- **√L:** if daily errors are independent, variances add over L days, so the standard deviation grows with √L. Seven days of error ≈ 15.25 × 2.646 ≈ 40 units.
- **Constraints** apply in order: case pack → MOQ → warehouse capacity → shelf life. Whichever cuts the order is reported in `binding_constraints`.
- **Shelf-life cap:** never hold more than 85% of what can be sold before expiry.

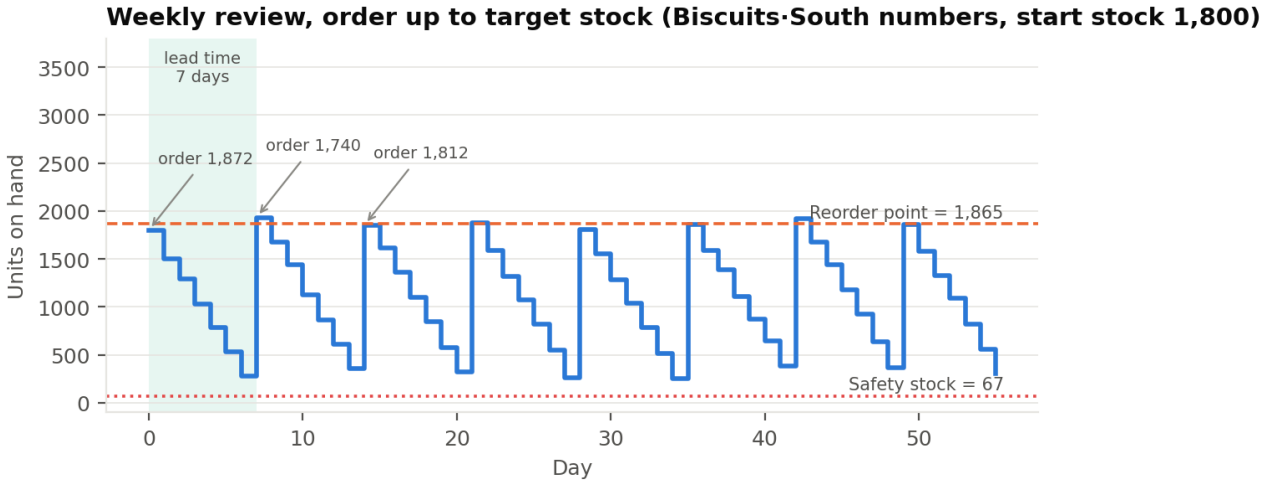
11.3 Worked example (the dashboard's default scenario)

Premium Biscuits 300 g, South, 28-day horizon, 10% promotion, current stock 1,800, lead time 7, service level 95%, case pack 12, MOQ 120, capacity 5,000, shelf life 60.

Step	Calculation	Result
Average daily forecast	mean of 28 recursive forecasts	256.9
Lead-time demand	256.9 × 7	1,798
Safety stock	1.645 × 15.25 × √7 = 1.645 × 15.25 × 2.646	66.4 → 67
Reorder point	1,798 + 67	1,865
Target stock	256.9 × (7 + 7) + 66.4	3,663
Unconstrained order	ceil(3,663 − 1,800)	1,863
Case-pack rounding	ceil(1,863 / 12) × 12 = 156 × 12	**1,872**
MOQ check	1,872 ≥ 120	OK
Capacity check	floor((5,000 − 1,800) / 12) × 12 = 3,192	OK
Shelf-life check	0.85 × 256.9 × 60 − 1,800 ≈ 11,300	OK
Stockout risk	1 − Φ((1,800 − 1,798) / (15.25 × 2.646))	**≈ 48%**

Interpretation: current stock (1,800) roughly equals demand over the next 7-day lead time, so there is about a coin-flip chance of running out before the new order arrives. Stock is below the

reorder point (1,865), so the system recommends ordering 1,872 units now.



Weekly review: each order lifts on-hand stock after the 7-day lead time; stock cycles above the safety-stock floor.

Upgrades to mention

Use SKU- or segment-specific σ (or the quantile forecast directly) instead of one global residual σ.

Add lead-time variability: $\sigma_{LT} = \text{sqrt}(L \cdot \sigma_d^2 + d^2 \cdot \sigma_L^2)$.

Use the newsvendor critical ratio (underage cost / (underage + overage cost)) to choose the service level from economics.

Optimise jointly across SKUs with shared truck or warehouse capacity (linear or integer programming).

11.4 Tested business properties

- More stock never increases the recommended order, and always lowers stockout risk (monotonicity test).
- Orders are always multiples of the case pack and never exceed capacity.

12. Promotion uplift with a T-learner

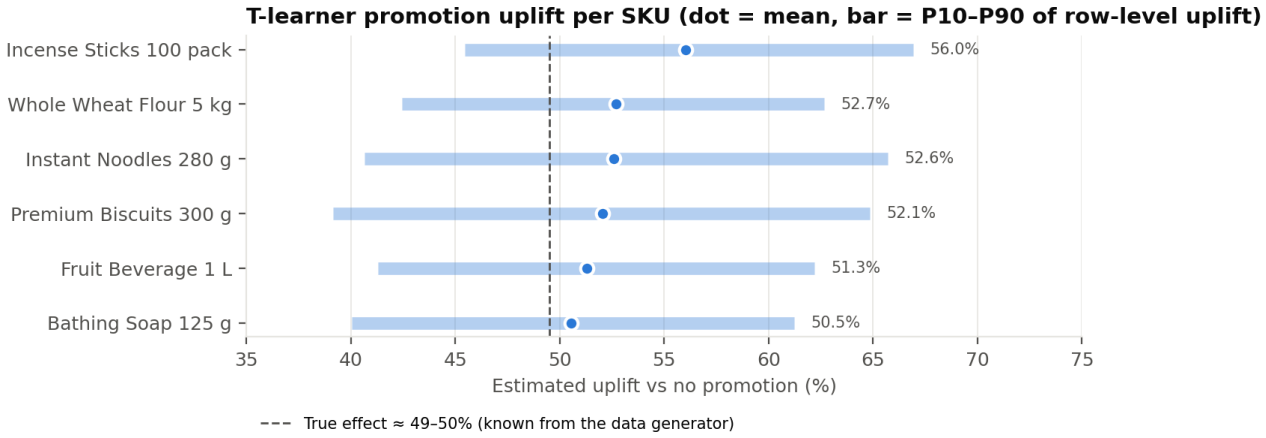
12.1 The question and the trap

“How much extra demand does a promotion create?” The naive answer compares the average demand on promotion days (177) with non-promotion days (117) → +51.6%. But promotions are **not random**: they happen more often Friday–Sunday, and weekends already sell 12% more. That is **confounding** — part of the naive difference is a weekend effect.

12.2 The T-learner (Two-model learner)

1. Split data at 90 days before the end. Train on the earlier part; score on the last ~91 days.
2. Train model T on **promotion days only** and model C on **non-promotion days only**.
3. Both use pre-promotion context only: SKU, category, region, channel, weather, calendar, lags and rolling stats. Price, discount and marketing are deliberately excluded because they are part of the treatment itself.
4. For every scoring row predict both: $\hat{Y}_{\text{promo}}$ and $\hat{Y}_{\text{no-promo}}$. The difference is the individual uplift (CATE estimate).
5. Average per SKU; report the 10th–90th percentile of row-level uplift as a range.

```
model_t = fit(treated[PROMO_FEATURES], treated.units_demand)
model_c = fit(control[PROMO_FEATURES], control.units_demand)
uplift_pct = (model_t.predict(X) - model_c.predict(X)) / max(model_c.predict(X), 1) × 100
```



12.3 Sanity check against the known truth

Because the data is synthetic, the true effect is known. The average discount is about 13.5%, so the generator's promo multiplier is 1 + 0.28 + 0.012 × 13.5 ≈ 1.44. Promo days also get higher marketing spend (1,450 vs 520), adding about 3.6% → true uplift ≈ **49–50%**. The T-learner estimates 50.5–56.0%, so it recovers the effect within a few points.

Be careful with the causal claim

This is an **adjusted association**, not proven causality. It controls only for measured confounders; unmeasured ones (retailer displays, competitor actions) may remain.

Assumptions needed for a causal reading: no unmeasured confounding, overlap (similar contexts with and without promotion), and SUTVA (no spill-over).

Stronger methods: randomised A/B or geo tests, propensity-score weighting, doubly robust / DR-learner, X-learner, double ML (EconML), difference-in-differences or synthetic control.

T-learner weakness: the treated model has far fewer rows (14% of days), so its noise flows straight into the uplift estimate.

13. Monitoring and governance

13.1 Population Stability Index (PSI)

```
edges   = deciles of the reference (training) distribution, outer edges set to ±inf
ref%    = share of reference values in each bin;   cur% = share of current values
PSI     = Σ (cur% − ref%) × ln(cur% / ref%)
```

PSI	Status in code	Action
< 0.10	stable	No action
0.10 – 0.25	watch	Investigate; check accuracy and data pipeline
> 0.25	retrain	Retraining investigation (not automatic deployment)

Result: target PSI between train and holdout = **0.081 → stable**. PSI is a symmetric divergence (the sum of the KL divergences in both directions). In production, compute it per feature and per period, not only on the target.

13.2 Isolation Forest anomaly detection

Isolation Forest builds random trees that split on random features at random values. Anomalies are rare and different, so they get isolated in **fewer splits** (shorter average path length). Here it runs on [residual, |residual|, discount_pct, stockout_flag] for each holdout row, with 180 trees and contamination = 3.5%.

> **Honest caveat**
> It is fitted and scored on the same 1,344 holdout rows with contamination = 0.035, so the 3.57% anomaly rate (48 rows) is **set by construction**, not discovered. In production: fit on a reference window, score new data, and alert on rate changes and individual high scores.

13.3 Retraining and promotion gates

- Triggers: scheduled retrain, PSI above threshold, rising wMAPE or bias, coverage falling below target, business changes (new SKU, new channel).
- Champion/challenger: a new model is promoted only if it beats the current champion **and** the baseline on rolling backtests.
- Human in the loop: the model card states that outputs are decision support; a planner approves orders.

13.4 Model card

docs/MODEL_CARD.md documents intended use, out-of-scope use (no autonomous ordering, no causal claims), data, evaluation protocol, metrics and limitations. Mentioning model cards signals responsible-AI maturity.

14. The Planning Copilot

The copilot answers planner questions in natural language, but it is **not an LLM**. It is a deterministic, keyword-routed answer engine grounded in the project's own outputs, and it cites its sources.

If the question contains…	It answers from…
promotion / discount / uplift	promotion_uplift.csv: average uplift, units/day, P10–P90 range, plus a non-causal disclaimer
accurate / performance / backtest / reliable	metrics.json + rolling_backtests.csv: wMAPE, MAE improvement, bias, fold average
risk / stockout / inventory / order / reorder	The live scenario's policy: order, reorder point, safety stock, risk, constraints
anything else	A planning summary of the current forecast and order

How to describe it honestly

“It is a grounded, deterministic Q&A; layer: every number comes from a model artifact, so it cannot hallucinate. The natural upgrade is an LLM with retrieval (RAG) or tool-calling over the same artifacts, with the deterministic answers kept as guardrails.” Do **not** call it generative AI.

15. Serving, testing and delivery

15.1 FastAPI (api.py)

Endpoint	Purpose
GET /health	Liveness check for load balancers; returns the model training timestamp
GET /metadata	Supported SKUs and regions, plus the model metrics
POST /forecast	Runs a scenario; returns the daily forecast with interval, the inventory policy and the explanation

```
POST /forecast
{"sku": "FOOD-BISC-300G", "region": "South", "horizon": 28, "promotion": true, "discount_pct": 10,
 "current_stock": 1800, "lead_time_days": 7, "service_level": 0.95, "case_pack": 12,
 "minimum_order_qty": 120, "warehouse_capacity": 5000, "shelf_life_days": 60}
```

Pydantic validates every field (horizon 7–42, discount 0–30, lead time 1–30, service level 0.80–0.995 ...). Invalid input returns 422 automatically; an unknown SKU or region returns 400. The model is loaded once at start-up, not per request.

15.2 Gradio dashboard (app.py)

A KPI header (wMAPE, improvement, coverage, drift), scenario controls (SKU, region, horizon, promotion, discount, stock, lead time, service level, plus a constraints accordion) and six tabs: Inventory decision, Explainability, Forecast data, Model governance, Promotion intelligence, Planning copilot.

15.3 Tests (tests/test_core.py)

Test	What it protects
test_generated_data_passes_contract	Row count, no missing target, demand ≥ sales
test_lag_features_only_use_prior_dates	No leakage: lag_1 equals the previous day; target not in FEATURES
test_more_stock_never_increases_order	Business logic monotonicity
test_operational_constraints_are_applied	Case-pack multiple, capacity cap
test_forecast_intervals_are_ordered	lower ≤ forecast ≤ upper, non-negative
test_saved_model_beats_baseline	Model regression gate: MAE below baseline, wMAPE below 15%
test_advanced_artifacts_and_copilot	Artifacts exist; copilot routes inventory questions correctly

These include **data-science tests** (leakage, quality gates), not only software tests — a point worth stressing.

15.4 CI, Docker, reproducibility

- GitHub Actions: on every push and PR, install requirements on Python 3.12, run unit tests and compile all modules.
- Docker: python:3.12-slim image, installs requirements, exposes port 7860 and runs the dashboard.
- Reproducibility: fixed seed 42 for data and models; metrics are computed by train.py, not typed by hand; artifacts are saved with joblib.

16. The 5-minute live demo script

1. “The header shows quality on unseen future data: 8.6% wMAPE, 56% better than the baseline, PSI stable.”
2. Select **Premium Biscuits 300 g, South**, 28-day horizon. “Teal is history, the purple line is the forecast, the shaded band is the 80% interval.”
3. Toggle the 10% promotion. “The explanation compares this with a no-promotion counterfactual for the same day.”
4. Set current stock to 1,800 and lead time to 7. “The system recommends 1,872 units — case-pack rounded, under capacity and shelf-life limits — and shows about 48% stockout risk before replenishment.”
5. Lower capacity or shelf life to show a **binding constraint** cutting the order.
6. Promotion intelligence tab: “The T-learner estimates about 52% uplift; I state clearly that this is not causal proof.”
7. Copilot: ask “Why should I reorder?” — the answer is grounded with sources.
8. Model governance: three rolling backtests, bias, anomaly rate, human review.

17. Critical review: weak spots and how to answer them

Strong candidates raise limitations themselves. These were found by reading the code; each has a ready answer.

Weak spot	Detail	Your answer
The 8.58% is one-step-ahead.	In the holdout, lag features use real past demand. In the dashboard, forecasts are recursive, so error grows with horizon.	“The holdout measures next-day accuracy. My next step is a recursive backtest reporting wMAPE by horizon day, or switching to direct multi-horizon models.”
Synthetic data is easy.	Noise is 10% of expected demand, so the best possible wMAPE is about 8% (mean absolute value of the noise ≈ 0.8 × 10%). The model is near that floor.	“On real SKU-store-day data I would expect 20–40% wMAPE. Being near the theoretical noise floor here tells me the model learned the signal.”
Safety stock uses one global σ.	residual_std is pooled over all SKUs; the P10/P90 bands are shown but not used in the order calculation; lead-time variability is ignored.	“I would use segment-level σ or quantile forecasts directly, and add supplier lead-time variance.”
Anomaly rate is circular.	Isolation Forest is fit and scored on the same rows with contamination 3.5%.	“It is a demo of the monitoring hook; in production I'd fit on a reference window and score new data.”
Zero importance for promotion and holiday.	Redundancy with discount_pct; holiday constant in the holdout window.	“Permutation importance depends on the evaluation window and splits credit between correlated features; I'd add SHAP and drop the redundant flag.”
Interval under-coverage (77% vs 80%).	Upper tail too low: 13.5% of actuals above P90.	“Conformal calibration on a recent window, monitored per category.”
Future exogenous inputs are assumptions.	Weather = recent average; marketing fixed; holiday rule differs from training.	“In production these come from weather forecasts and the promotion calendar, aligned to the training definitions.”
Lost-sales correction is simple.	Ignores promotion or weekend context on stockout days.	“Model-based imputation or a censored-likelihood model.”
Uplift is not causal.	Adjusts only for observed context.	“For causal claims, experiments or doubly robust estimators with overlap checks.”
Copilot is rule-based.	Keyword routing, no language model.	“Deterministic by design for trust; RAG or tool-calling is the upgrade path.”
Minor details.	Early stopping uses a random 12% of training rows rather than the latest dates; MODEL_CARD.md still says constraints are omitted (outdated since v2); no gap between train and test windows; no hyperparameter tuning was logged.	“I would use time-ordered early stopping, keep docs in sync with releases, and tune with time-series cross-validation and Optuna.”

18. Interview question bank

A. Problem and data

Q. Why is the target units_demand and not units_sold?

Sales are censored by stock. On stockout days sales understate demand. Training on sales teaches the model to under-forecast, which causes more stockouts — a feedback loop. The target is corrected on stockout days.

Q. Why synthetic data?

To build an end-to-end system without confidential data. The generator has realistic drivers (seasonality, trend, promotions, weather, festivals, stockouts), and because the truth is known I can validate methods — for example, the T-learner recovered the true ~50% uplift.

Q. How would you adapt it to real data?

Replace data.py with ERP/POS ingestion, add data contracts and freshness checks, join real holiday, weather and promotion calendars with point-in-time correctness, go to store level with hierarchical reconciliation, and expect much higher wMAPE.

Q. What is the forecasting grain and why?

SKU × region × day, because replenishment decisions are made per product per distribution region, and daily granularity captures weekday and promotion effects.

B. Features and leakage

Q. What is data leakage and how did you prevent it?

Leakage is when training features contain information unavailable at prediction time. Prevention: groupby(sku, region) + shift(1) before every lag and rolling calculation, a time-based split, rolling-origin backtests, and a unit test asserting lag_1 equals the previous day's target.

Q. Why lags 1, 7, 14, 28?

1 = momentum; 7, 14, 28 = same weekday 1–4 weeks ago, which captures weekly seasonality. Rolling means smooth noise; rolling std captures volatility.

Q. Why sin/cos encoding?

It makes cyclical features continuous — Sunday sits next to Monday.

Q. Which features matter most?

rolling_mean_28 and discount_pct (each adds about 10–11 units of MAE when shuffled), then rolling_mean_7 and day_of_week.

C. Modelling

Q. Explain gradient boosting.

An additive ensemble of shallow trees, each fitted to the negative gradient of the loss (the residuals for squared error) of the current ensemble, added with a small learning rate. Histogram binning makes split-finding fast.

Q. Boosting vs bagging (random forest)?

Bagging trains deep trees independently in parallel and averages them to reduce variance. Boosting trains shallow trees sequentially, each correcting the previous ones, to reduce bias. Boosting is usually more accurate on tabular data but more sensitive to hyperparameters.

Q. How do you get prediction intervals?

Train separate quantile models (q = 0.1 and 0.9) with the pinball loss. Sort the outputs so lower ≤ point ≤ upper. Alternatives: conformal prediction, bootstrapped residuals, NGBoost, or probabilistic deep models.

Q. How did you prevent overfitting?

Shallow trees (24 leaves), at least 28 rows per leaf, L2 regularisation, a small learning rate, early stopping, a time-based holdout and rolling backtests.

Q. Why one global model instead of 24 local models?

It shares information across series (weekend or promotion effects), needs less data per series, is easier to maintain, and handles new series through category and region features.

Q. Why not deep learning?

With 24 series and about 450 days, boosting is faster, more accurate and more explainable. I would benchmark DeepAR, TFT or N-BEATS with thousands of series and long history.

Q. How would you tune hyperparameters?

Optuna or random search, scored on rolling-origin time-series cross-validation (never random K-fold), optimising wMAPE with a bias constraint.

D. Evaluation

Q. Why wMAPE over MAPE?

MAPE explodes for near-zero actuals and weights every row equally. wMAPE = Σ|e|/Σy weights by volume, which matches business impact.

Q. Why care about bias if wMAPE is good?

wMAPE uses absolute errors, so it shows how big errors are but not which direction they lean. Inventory costs are asymmetric: a consistent under-forecast (−1.5%) means repeated stockouts. Bias shows the direction, so it is tracked separately.

Q. What does 77% coverage tell you?

The 80% interval is slightly too narrow — mostly on the upside (13.5% of actuals above P90). Calibrate with conformal prediction.

Q. What is rolling-origin backtesting?

Repeatedly train on all data before a cut-off and test on the next window, moving the cut-off forward. Here: 3 × 28-day folds, wMAPE 9.74 / 8.60 / 8.61%, always beating the baseline by 49–58%.

Q. Is 56.5% improvement over lag-7 realistic?

On synthetic data with strong promotion effects, yes, because lag-7 cannot anticipate promotions. On real data, improvements of 10–30% over a seasonal naive baseline are more typical.

E. Inventory and business

Q. Derive safety stock.

Assume forecast errors are normal with standard deviation σ per day and independent across days. Over L days the error standard deviation is σ√L. To cover the service-level quantile, hold $z \cdot \sigma \cdot \sqrt{L}$, where $z = \Phi^{-1}$(service level).

Q. What does service level 95% mean?

A 95% probability of not stocking out during a replenishment cycle (cycle service level). A fill rate — the share of demand served — is a different metric.

Q. Why is stockout risk 48% in the demo?

Stock (1,800) ≈ lead-time demand (1,798), so the z-score is about 0 → roughly a 50% chance that demand exceeds stock before replenishment arrives.

Q. How does the forecast create business value?

Lower error → smaller σ → less safety stock for the same service level → less working capital and fewer stockouts. Measure with forecast value-add (FVA) against the baseline and planner overrides.

Q. How would you choose the service level?

From economics: the newsvendor critical ratio Cu / (Cu + Co), where Cu = margin lost per unit short and Co = holding or expiry cost per unit over. High-margin, long-shelf-life items warrant higher service levels.

F. Promotion and causality

Q. What is a T-learner?

A meta-learner for heterogeneous treatment effects: fit one outcome model on treated rows and one on control rows; the uplift is the difference of their predictions. Alternatives: S-learner (one model with treatment as a feature), X-learner, DR-learner, causal forests.

Q. Why exclude price and discount from uplift features?

They are part of the treatment. Conditioning on them would absorb the effect you want to measure (post-treatment or mediator bias).

Q. Is the 52% uplift causal?

No. It is adjusted for observed context only. For causality: randomised tests, or observational methods with an explicit no-unmeasured-confounding assumption and overlap checks.

G. MLOps and production

Q. How do you detect drift?

PSI per feature and on the target (0.081 here, stable), plus monitoring of wMAPE, bias, interval coverage, data freshness, null rates and unseen categories.

Q. When would you retrain?

On a schedule plus triggers (PSI, accuracy decay, coverage loss, business change). Promote only if the challenger beats the champion and the baseline on backtests; keep a human approval step.

Q. How would you scale to 10,000 SKUs × 500 stores?

Feature pipelines in SQL or Spark, a global LightGBM model per category, batch scoring orchestrated by Airflow, MLflow tracking and registry, a feature store, hierarchical reconciliation (MinT), and monitoring dashboards.

Q. How do you handle a new product (cold start)?

Category, region, price and pack-size features; similar-product analogues; a category-level prior that blends toward the SKU's own history as data accrues.

Q. What is train/serve skew and how did you avoid it?

Different transformations in training and serving. Avoided by saving the full scikit-learn Pipeline (encoder + model) in one joblib bundle that both the API and the UI load.

H. Behavioural

Q. What was the hardest part?

Recognising that sales are censored and that metrics must reflect decisions — moving from “accurate forecast” to “correct order with known risk.”

Q. What would you do with two more weeks?

Recursive or direct multi-horizon backtests, conformal calibration, SHAP explanations, segment-level safety stock, hierarchical reconciliation and an MLflow registry.

19. Glossary of concepts

Concept	One-line definition
Censoring	Observed value is capped (sales ≤ stock), so it is a lower bound on the true value.
Data leakage	Training features contain information not available at prediction time.
Lag feature	The target's value k periods earlier.
Rolling feature	Statistic over a trailing window, computed after shifting by one period.
Gradient boosting	Sequential ensemble of trees, each fitting the loss gradient of the current model.
Quantile regression	Predicts a conditional quantile instead of the mean, using the pinball loss.
Pinball loss	$q \cdot e$ if $e \geq 0$, $(q - 1) \cdot e$ if $e < 0$, where e = actual − prediction.
Prediction interval	Range expected to contain a future observation with a given probability.
Coverage	Fraction of actuals inside the interval; should match the nominal level.
Conformal prediction	Distribution-free method to calibrate intervals using held-out residuals.
wMAPE	$\Sigma\lvert\text{actual} - \text{forecast}\rvert / \Sigma$ actual.
Bias	$\Sigma(\text{forecast} - \text{actual}) / \Sigma$ actual.
Seasonal naive	Forecast = value one season ago (here 7 days).
Rolling-origin backtest	Repeated forward-in-time train/test evaluation.
Recursive forecasting	Feed predictions back as lags to forecast further ahead.
Direct forecasting	A separate model for each forecast horizon.
Permutation importance	Increase in error when one feature is shuffled.
SHAP	Game-theoretic additive attribution of a single prediction to features.
Counterfactual	Prediction under an altered input (promotion off) to isolate an effect.
T-learner	Uplift = model(treated) − model(control).
Confounding	A variable affects both treatment and outcome (weekend → promotion and demand).
PSI	$\Sigma(\text{cur} - \text{ref}) \cdot \ln(\text{cur}/\text{ref})$ over bins; a drift measure.
Isolation Forest	Anomaly detector: anomalies need fewer random splits to isolate.
Safety stock	$z \cdot \sigma \cdot \sqrt{L}$ buffer against forecast error during lead time.
Reorder point	Lead-time demand + safety stock.
Order-up-to policy	At each review, order enough to reach a target level.
Newsvendor	Single-period stocking model; optimal quantile = Cu / (Cu + Co).
Hierarchical reconciliation	Make SKU forecasts sum consistently to category and total forecasts.
Model card	Document of a model's intended use, data, metrics and limitations.
Train/serve skew	Mismatch between training and serving transformations.

20. One-page cheat sheet and self-test

Numbers to memorise

Item	Value	Item	Value
Rows	11,520 (6 SKU × 4 region × 480 d)	Train / valid rows	9,504 / 1,344
Holdout window	56 days, 2026-05-06 → 06-30	Features	23 (4 categorical + 19 numeric)
wMAPE	8.58%	Baseline wMAPE	19.74%
MAE / RMSE	11.53 / 15.38	Baseline MAE	26.52
MAE improvement	56.51%	Bias	−1.50%
80% coverage	77.08%	PSI	0.081 (stable)
Backtests	9.74 / 8.60 / 8.61% → mean 8.98%	Anomaly rate	3.57% (by construction)
Top features	rolling_mean_28, discount_pct	Uplift	≈ 50.5–56% (truth ≈ 49–50%)
Model	HistGradientBoosting × 3 (point, P10, P90)	Seed	42
Demo order	1,872 units (ROP 1,865, SS 67)	Demo stockout risk	≈ 48%

Key formulas

```
wMAPE  = Σ|y − ŷ| / Σy                Bias   = Σ(ŷ − y) / Σy
PSI    = Σ (c − r)·ln(c / r)            Pinball_q(e) = q·e if e ≥ 0 else (q − 1)·e,  e = y − ŷ
SS     = z·σ·√L                         ROP    = d̄·L + SS
Target = d̄·(L + R) + SS                  Order  = ceil((Target − Stock) / pack) · pack
Stockout risk = 1 − Φ((Stock − d̄·L) / (σ·√L))
Uplift = f_T(x) − f_C(x)
```

Self-test — answer aloud without looking

1. Why is the target units_demand, and how is it computed on stockout days?
2. Which line prevents leakage in rolling features, and why does it work?
3. Why a time-based split, and what is rolling-origin backtesting?
4. Explain gradient boosting and the pinball loss in two sentences each.
5. What are wMAPE, bias and coverage here, and what does each tell a planner?
6. Derive safety stock and explain the √L.
7. Walk through the 1,872-unit order from the forecast.
8. Why are promotion and holiday importances zero?
9. What is a T-learner, why exclude price from its features, and is the result causal?
10. Name three limitations and the fix for each.

Final advice

Say “synthetic” and “decision support” early and confidently — honesty about scope builds trust.

Every time you state a metric, state the baseline and the business meaning next to it.

If you don't know something, say how you would find out (which experiment, which metric).

Appendix: resume bullets (accurate wording)

- Built an end-to-end FMCG demand forecasting and inventory decision platform (Python, scikit-learn, FastAPI, Gradio, Docker) over 11,520 synthetic SKU-region-day records with stockout-censoring correction.
- Achieved 8.58% holdout wMAPE and 56.5% lower MAE than a seasonal-naive baseline using leakage-safe lag and rolling features, a strict 56-day time holdout and three rolling-origin backtests (mean wMAPE 8.98%).
- Delivered 80% prediction intervals via quantile gradient boosting and converted forecasts into case-pack, MOQ, capacity and shelf-life-constrained replenishment orders with service-level safety stock.
- Estimated promotion uplift with a context-adjusted T-learner, and added PSI drift monitoring, Isolation Forest anomaly flags, permutation-importance explanations, a grounded rule-based planning assistant, unit tests and CI.